

ECE 205 — Practice Exam 1 (Ungraded)

This is a self-assessment tool for **Midterm 1**. It mirrors the real exam in format, scope, length, and difficulty, but the questions are different. You may attempt to try and answer these your self with no notes and a timer (40 minutes). If you have questions before the real exam, please ask the instructor.

You should write out your answers and bring them to the minderm.

Scope (same as Midterm 1)

Classes & encapsulation · constructors/destructors · inheritance vs. composition · abstract classes & interfaces · the Open/Closed Principle · arrays (1D/2D) · design patterns (Object Pool, Singleton, Factory Method, Strategy, RAII)

Not covered here (or on the midterm): operator overloading, recursion, linked lists/trees, templates, exact C++ syntax.

Questions

Section A — Classes & Encapsulation

A1. What problem does encapsulation solve? Answer without using the word "hide" or "private."

A2. A **Timer** class stores **secondsElapsed** as a public **int**. Any part of the program can set it directly, including to negative values. Explain why this is a design flaw and describe a fix.

A3. What is the practical difference between assigning a member variable in the constructor's body versus using an initialization list? (Conceptually — you don't need exact syntax.) Name one situation where an initialization list isn't just preferred but required.

A4. A class dynamically allocates a resource in its constructor and correctly frees it in its destructor. A programmer then copies an object of this class using the default (compiler-generated) copy behavior. What problem is likely to occur when both the original and the copy are destroyed? (You don't need to know the fix by name — just explain what goes wrong.)

Section B — Inheritance & Composition

B1. A **Student** class and a **Person** class are related, as are a **Student** class and a **Schedule** class. Which relationship (**Student/Person** or **Student/Schedule**) should be modeled with inheritance, and which with composition? Justify both.

B2. Explain what changes about a base class's **public** members when a derived class uses **protected** inheritance instead of **public** inheritance. Who is affected by this change?

B3. What is the difference between **hierarchical inheritance** and **multiple inheritance**? Give a short example of each.

B4. Give one advantage composition has over inheritance when your design might need to change later. Be specific — "more flexible" alone is not a full answer.

Section C — Abstract Classes & Interfaces

C1. True or false, with a one-sentence justification: "An abstract class can have member variables and fully-implemented (non-pure) methods, as long as it also has at least one pure virtual method."

C2. Explain the difference between a *concrete* class and an *abstract* class in terms of what each one guarantees about its methods.

C3. A base class `Employee` declares `virtual double calculatePay() = 0;`. Two derived classes, `SalariedEmployee` and `HourlyEmployee`, each implement it differently. Explain how a single function like `void printPaycheck(Employee* e)` can correctly print the right pay for either kind of employee without an `if/else` checking the type.

C4. Why is it considered bad practice to call a pure virtual function from within a base class's constructor or destructor?

C5. What are the two requirements for a class to count as a "pure interface" rather than just a general abstract class?

Section D — The Open/Closed Principle

Note: this topic was just added to the course notes — expect direct questions on it on Midterm 1.
Read [04e-OpenClosedPrinciple.md](#) before attempting these.

D1. State the Open/Closed Principle in your own words, and explain why "open for extension" and "closed for modification" aren't actually a contradiction once you know the trick.

D2. A payment processing function is written as `if (method == "credit") ... else if (method == "paypal") ...`. Explain why this violates the Open/Closed Principle, using the same reasoning style as the shape/area example in lecture.

D3. Redesign the function from D2 using an abstract class. What would the abstract class be called, what pure virtual method would it declare, and what — specifically — would no longer need to change when a new payment method is added?

Section E — Arrays

E1. A student declares `int scores[30];` and later writes a loop `for (int i = 0; i <= 30; i++)`. What's wrong with this loop, and what will happen when it runs?

E2. Arrays are described as being passed "by reference" to functions in C/C++, even without using the `&` reference syntax. Explain what this actually means about how the array is passed, and why a function receiving an array parameter also needs to be told the array's size separately.

E3. Given `int grid[4][6];`, describe in words how the element `grid[2][3]` is located in memory, referencing row-major order.

Section F — Design Patterns

F1. For each scenario, name the design pattern that fits best (Singleton, Object Pool, Factory Method, or Strategy), and explain why in one sentence:

- a. An application needs one, and only one, **Logger** object that every module writes to.
- b. A particle system spawns and destroys hundreds of short-lived particle effects per second, and constant allocation is causing frame drops.
- c. A checkout system needs to support multiple payment methods (credit card, PayPal, store credit) that can be swapped at runtime based on what the customer selects.
- d. A UI toolkit needs to create different **Button** subtypes (**RoundButton**, **SquareButton**) based on a theme setting, without the UI code depending on those concrete classes directly.

F2. Explain, in your own words, why RAII is sometimes described as a design pattern even though it's really "just" using constructors and destructors correctly. What problem does it solve that manual **new/delete** doesn't solve on its own?

F3. Design scenario: You're building a notification system that can send messages via **Email**, **SMS**, or **PushNotification**. You want to add new delivery channels later without touching existing code, and you want calling code to just say "send this message" without caring which channel is used underneath. Sketch (in words or a simple diagram) the abstract class/interface you'd define and how a Factory Method would fit into this design.